

Uniwersytet Wrocławski
Wydział Fizyki i Astronomii

Oliwer Urbaniak

System zarządzania pracą drukarki 3D w laboratorium
studenckim.

Praca inżynierska na kierunku
Informatyka Stosowana i Systemy Pomiarowe

Opiekun
dr Bartosz Brzostowski

Wrocław, 19 maja 2026

Spis treści

1	Wprowadzenie	8
1.1	Motywacja i cel pracy	8
1.2	Zakres pracy	8
1.3	Struktura pracy	9
2	Analiza wymagań i przegląd rozwiązań	10
2.1	Wymagania funkcjonalne	10
2.2	Wymagania нефункционалне	10
2.3	Przegląd istniejących rozwiązań	12
2.3.1	Octoprint	12
2.3.2	Mainsail i Fluidt	13
2.3.3	Rozwiązania komercyjne	13
2.3.4	Wnioski	13
3	Wybór technologii	14
3.1	Języki programowania	14
3.1.1	TypeScript / Node.js	14
3.1.2	Python	14
3.2	Framework webowy - Express.js	14
3.3	Baza danych - Azure Cosmos DB	14
3.4	Przechowywanie plików - Azure Blob Storage	15
3.5	Kolejkowanie - Azure Service Bus	15
3.6	Sterowanie urządzeniem - Azure IoT Hub	15
3.7	Frontend - React i Vite	16
3.8	Kontenery - Docker i Docker Compose	16
3.9	CAD - SolidWorks	17
3.10	Porównanie wybranych alternatyw	17
4	Architektura systemu	18
4.1	Ogólna struktura	18
4.2	Diagram architektury	18

4.3	Model danych	20
4.3.1	Użytkownik (kontener users)	21
4.3.2	Zadanie do wydruku (kontener jobs)	21
4.3.3	Token odświeżania (kontener refresh-tokens)	22
4.4	Cykl życia zadania do wydruku	22
4.5	Bezpieczeństwo i uwierzytelnianie	24
5	Implementacja serwisów backendowych	26
5.1	Auth Service	26
5.1.1	Opis ogólny	26
5.1.2	Endpointy API	26
5.1.3	Kluczowe fragmenty implementacji	26
5.1.4	Weryfikacja e-mail	26
5.2	User Service	26
5.2.1	Opis ogólny	26
5.2.2	Endpointy API	26
5.2.3	Mechanizm autoryzacji	26
5.3	Files Service	26
5.3.1	Opis ogólny	26
5.3.2	Przepływ przesyłania pliku	26
5.4	Queue Service	26
5.4.1	Opis ogólny	26
5.4.2	Listener kolejki	26
5.4.3	Endpointy HTTP API	26
5.5	Printer Service	26
5.5.1	Opis ogólny	26
5.5.2	Harmonogram	26
5.5.3	Monitor gotowości	26
5.5.4	Listener telemetrii	26
5.5.5	Enpointy HTTP API	26
5.6	Video Service	26

6	Implementacja frontendu	27
6.1	Struktura aplikacji	27
6.2	Klient API	27
6.3	Routing i ochrona tras	27
6.4	Ekrany aplikacji	27
6.4.1	Strona główna	27
6.4.2	Dashboard użytkownika	27
6.4.3	Upload i planowanie	27
6.4.4	Kontrola druku	27
6.4.5	Panel administracyjny	27
6.5	Internacjonalizacja	27
6.6	Obsługa motywu	27
7	Agent Raspberry Pi - AddiPi Agent	28
7.1	Opis ogólny	28
7.2	Architektura agenta	28
7.3	Połączenie z IoT Hub	28
7.4	Obsługiwane metody bezpośrednie	28
7.5	Przebieg obsługi metody startPrint	28
7.6	Telemetria	28
7.7	Komunikacja z OctoPrint	28
8	Obudowa urządzenia - projekt CAD	29
8.1	Cel i zakres projektu mechanicznego	29
8.2	Moduł CASE - obudowa Raspberry Pi z kamerą	29
8.2.1	Podstawa projektowa	29
8.2.2	Zakres modyfikacji	29
8.2.3	Mocowanie kamery	29
8.2.4	Struktura plików CASE	29
8.3	Moduł STAND - podstawka regulowana	29
8.4	Złożenie całości	29
8.5	Parametry wydruku	29
8.6	Instrukcja montażu	29

9	Konfiguracja Raspberry Pi	30
9.1	Opis środowiska	30
9.2	Instalacja agenta	30
9.3	Automatyczne uruchamianie agenta przez systemd	30
9.4	Tunel Cloudflare - ekspozycja kamery	30
9.4.1	Instalacja cloudflared	30
9.4.2	Skrypt publikowania URL kamery	30
9.4.3	Cykliczne odnawianie URL	30
9.5	Konfiguracja crontab	30
9.5.1	Uzasadnienie parametrów	30
9.6	Problem z dostępnością sieci przy starcie	30
9.7	Podsumowanie konfiguracji Raspberry Pi	30
10	Wdrożenie i infrastruktura	31
10.1	Infrastruktura chmurowa	31
10.2	Inicjalizacja infrastruktury	31
10.3	Konteneryzacja - Dockerfiles	31
10.4	Docker Compose	31
10.5	Zmienne środowiskowe	31
10.6	Wdrożenie frontendu	31
10.7	Wdrożenie agenta	31
11	Testowanie systemu	32
11.1	Metodologia testowania	32
11.2	Testy jednostkowe - Auth Service	32
11.3	Testy integracyjne przez Postman	32
11.4	Test end-to-end - pełny przepływ druku	32
11.5	Testy wydajnościowe i obserwacje	32
12	Podsumowanie	33
12.1	Osiągnięcia	33
12.2	Napotkane trudności	33
12.3	Możliwości rozwoju	33
12.4	Wnioski	33

Załączniki	36
A Konfiguracja środowiska deweloperskiego	36
A.1 Wymagania	36
A.2 Pierwsze uruchomienie	36
B Struktura repozytoriów	36
C Endpointy API - podsumowanie	36

Streszczenie

Niniejsza praca inżynierska opisuje projekt, implementację oraz wykonanie systemu *AddiPi* - rozproszonego systemu zarządzania pracą drukarki 3D przeznaczonego dla laboratorium studenckiego. System umożliwia użytkownikom przesyłanie plików G-code, planowanie oraz monitorowanie zadań do wydruku, a administratorom - pełną kontrolę nad kolejką i użytkownikami.

Architektura systemu oparta jest na podejściu mikroserwisowym. Wyodrębniono sześć niezależnych serwisów backendowych: uwierzytelniania (Auth Service), zarządzania użytkownikami (User Service), obsługi plików (Files Service), kolejkowania zadań (Queue Service), sterowania drukarką (Printer Service) oraz agenta działającego na urządzeniu Raspberry Pi (AddiPi Agent). Całość uzupełnia aplikacja frontendowa zbudowana w React oraz serwis wideo do podglądu kamery.

Komunikacja między serwisami odbywa się za pośrednictwem protokołu HTTP/REST oraz chmurowej kolejki Azure Service Bus. Dane przechowywane są w Azure Cosmos DB, a pliki G-code w Azure Blob Storage. Sterowanie drukarką realizowane jest przez Azure IoT Hub przy użyciu bezpośrednich wywołań metod (Direct Methods) na Raspberry Pi, na którym działa oprogramowane OctoPrint.

System zrealizowano przy użyciu języków TypeScript (Node.js) i Python. Wdrożenie odbywa się w kontenerach Docker, koordynowanych plikiem Docker Compose. Interfejs użytkownika obsługuje uwierzytelnianie tokenem JWT, przesyłanie plików, planowanie zadań, podgląd kamery w czasie rzeczywistym oraz panel administracyjny.

Słowa kluczowe: mikroserwisy, drukarka 3D, OctoPrint, Raspberry Pi, Azure Iot Hub, Azure Cosmos DB, Docker, TypeScript, Python, REST API, kolejkowanie zadań

1 Wprowadzenie

1.1 Motywacja i cel pracy

Drukarki 3D stanowią obecnie standardowe wyposażenie laboratoriów akademickich, warsztatów studenckich oraz zakładów produkcyjnych zorientowanych na wytwarzanie przyrostowe. Urządzenia te, pomimo rosnącej dostępności i malejących cen, pozostają zasobem współdzielonym - ich liczba jest zwykle ograniczona, a czas drukowania pojedynczego modelu może wynosić od kilkunastu minut do wielu godzin. W środowisku akademickim oraz przemysłowym rodzi to naturalne problemy organizacyjne: konieczność ręcznego umawiania się na czas druku, brak przejrzystości stanu kolejki oraz niemożność zdalnego monitorowania postępu pracy.

Celem niniejszej pracy jest zaprojektowanie i implementacja systemu *AddiPi*, który automatyzuje proces zarządzania drukarką 3D w laboratorium studenckim. System pozwala użytkownikom na:

- przesyłanie plików G-code przez przeglądarkę internetową,
- planowanie druków na określony termin lub dodawanie ich do kolejki,
- śledzenie postępu druku w czasie rzeczywistym, wraz z podglądem kamery,
- otrzymywanie powiadomień o zakończeniu lub błędzie druku.

Administratorzy laboratorium zyskują natomiast narzędzia do zarządzania użytkownikami, przeglądania historii wszystkich zadań oraz sterowania kolejką i urządzeniami.

1.2 Zakres pracy

Praca obejmuje:

1. analizę wymagań systemu i wybór technologii,
2. projekt architektury mikroserwisowej,
3. implementację wszystkich komponentów backendowych i frontendowych,
4. konfigurację infrastruktury chmurowej (Microsoft Azure),
5. wdrożenie systemu przy użyciu kontenerów Docker,

6. testy poprawności działania.

1.3 Struktura pracy

Rozdział 2 zawiera analizę wymagań i przegląd istniejących rozwiązań. Rozdział 3 omawia wybrane technologie i uzasadnia decyzje projektowe. Rozdział 4 opisuje architekturę systemu. Rozdziały 5–7 szczegółowo omawiają implementację poszczególnych komponentów programowych. Rozdział 8 opisuje projekt mechaniczny obudowy w SolidWorks. Rozdział 9 omawia konfigurację Raspberry Pi, automatyzację przez cron i tunel Cloudflare. Rozdział 10 dotyczy wdrożenia i infrastruktury chmurowej Azure. Rozdział 11 przedstawia wyniki testów. Praca kończy się podsumowaniem w rozdziale 12.

2 Analiza wymagań i przegląd rozwiązań

2.1 Wymagania funkcjonalne

Na podstawie analizy potrzeb laboratorium studenckiego sformułowano następujące wymagania funkcjonalne:

Dla użytkowników:

- rejestracja z weryfikacją adresu e-mail (ograniczoną do domeny uczelni @uwr . edu . pl),
- logowanie z użyciem tokenów JWT (access token + refresh token),
- przesyłanie plików G-code (maksymalnie 50 MB, format . gcode),
- planowanie druku na wybrany termin lub dodanie do kolejki ogólnej,
- przeglądanie własnych zadań z filtrowaniem według statusu,
- podgląd postępu druku w czasie rzeczywistym (pasek postępu, temperatura dyszy i podłoża, obraz z kamery),
- anulowanie i ponowne uruchamianie własnych zadań.

Dla administratorów:

- pogląd i zarządzanie wszystkimi użytkownikami systemu,
- zmiana roli użytkownika (użytkownik/administrator),
- podgląd i zarządzanie wszystkimi zadaniami druku,
- potwierdzenie gotowości drukarki do kolejnego zlecenia,
- dostęp do metryk systemowych (liczba zadań według statusu).

2.2 Wymagania нефunkcjonalne

- **Skalowalność** - architektura powinna umożliwiać dodanie kolejnych drukarek bez przebudowy systemu.
- **Niezawodność** - utrata wiadomości z kolejki nie powinna powodować trwałej utraty zadania.

- **Bezpieczeństwo** - dostęp do API chroniony tokenami JWT, hasła przechowywane w formie szyfru bcrypt.
- **Responsywność interfejsu** - aplikacja frontendowa powinna działać zarówno na urządzeniach desktopowych, jak i mobilnych.
- **Czas odpowiedzi** - operacje API powinny odpowiadać w czasie poniżej 2 sekund w warunkach normalnego obciążenia.
- **Przenośność** - wszystkie komponenty opakowane w kontenery Docker, uruchamiane jednym poleceniem.

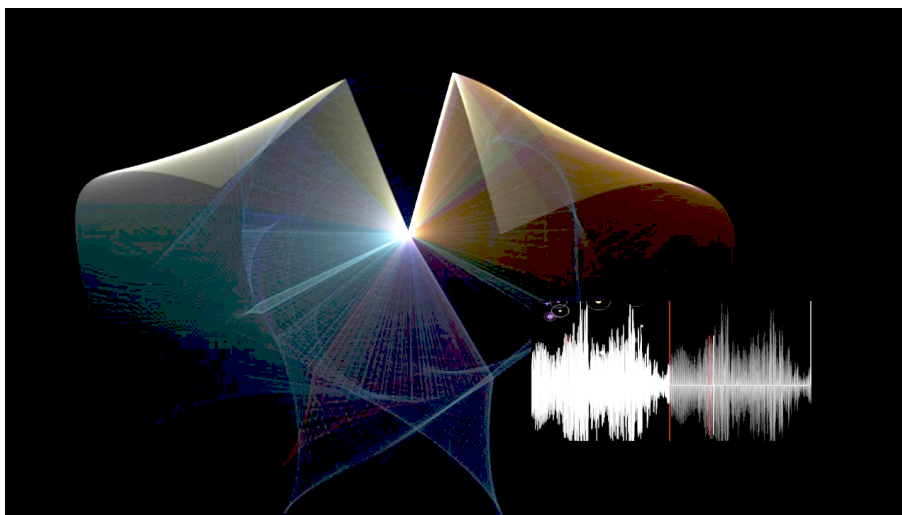
W swojej pracy możesz odnosić się do równań np. do równania (1) lub do rysunków, zarówno wektorowych, patrz rys. (1) jak i bitmapowych (2). Możesz też z powodzeniem cytować literaturę pojedynczo [5] lub kilka pozycji na raz [?, 1]. Jeśli chcesz to robić wygodniej załóż tzw. plik bibtex, ale możesz też pracować tak, jak w tym dokumencie.

$$\rho \left(\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \nabla) \vec{v} \right) = \rho \vec{f} - \nabla p + \mu \Delta \vec{v}, \quad (1)$$

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



Rysunek 1: Podpis rysunku, który jest obrazem wektorowym (EPS).



Rysunek 2: To jest rysunek drugi, kilkadziesiąt wahadeł podwójnych z synteza dźwięku.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. .. in the figure 2.2.

2.3 Przegląd istniejących rozwiązań

2.3.1 Octoprint

OctoPrint [4] jest najpopularniejszym oprogramowaniem do zarządzania drukarkami 3D. Działa jako serwer HTTP na Raspberry Pi i udostępnia interfejs webowy, API REST oraz system wtyczek. Umożliwia zdalne sterowanie drukarką, podgląd z kamery i monitorowanie postępu.

OctoPrint jest jednak przeznaczony dla jednego użytkownika zarządzającego jedną drukarką. Nie oferuje mechanizmu kolejkowania zadań wielu użytkowników, systemu uwierzytelniania opartego na rolach ani integracji z chmurą. W systemie AddiPi OctoPrint pełni rolę lokalnego sterownika drukarki, nad którym nadbudowano opisywaną w tej pracy warstwę zarządzania.

2.3.2 Mainsail i Fluidt

Mainsail i Fluidt to nowoczesne interfejsy webowe dla firmware Klipper. Podobnie jak OctoPrint, skierowane są do jednego użytkownika i nie przewidują obsługi kolejki wielu użytkowników.

2.3.3 Rozwiązania komercyjne

Oblio, Polar Cloud i Ultimaker Digital Factory to komercyjne platformy zarządzania flotą drukarek 3D w środowisku przemysłowym i edukacyjnym. Oferują zaawansowane funkcje, jednak wiążą się z kosztami subskrypcji i nie pozwalają na pełną kontrolę nad infrastrukturą - co w warunkach laboratorium akademickiego jest istotną przeszkodą.

2.3.4 Wnioski

Żadne z dostępnych rozwiązań o otwartym kodzie źródłowym nie łączy w sobie: systemu uwierzytelniania obsługującego wielu użytkowników, kolejkowania zadań z planowaniem czasowym, integracji z chmurą obliczeniową i sterowaniem przez IoT. System AddiPi wypełnia tę lukę, pozostając jednocześnie w pełni samodziśalający i wdrażany lokalnie.

3 Wybór technologii

3.1 Języki programowania

3.1.1 TypeScript / Node.js

Większość serwisów backendowych oraz aplikacja frontendowa zostały napisane w TypeScript [11] - nadzbiorze języka JavaScript dodającym statyczne typowanie. TypeScript znacznie ułatwia utrzymanie dużych baz kodu dzięki wykrywaniu błędów na etapie kompilacji, podpowiedziom IDE i samodokumentowaniu przez typy.

Node.js [13] wybrano ze względu na asynchroniczny, nieblokujący model wejścia-wyjścia, który sprawdza się w serwisach obsługujących wiele równoległych połączeń HTTP oraz komunikację z zewnętrznymi usługami (baza danych, kolejka, IoT Hub).

3.1.2 Python

Agent działający na platformie Raspberry Pi został zaimplementowany w języku Python [15]. Biblioteki `azure-iot-device` oraz `azure-storage-blob` są w tym środowisku powszechnie dostępne i szczegółowo udokumentowane. Python stanowi standardowe rozwiązanie w przypadku aplikacji uruchamianych na systemach wbudowanych oraz mikrokomputerach jednoukładowych (ang. *Single-Board Computer*, SBC).

3.2 Framework webowy - Express.js

Wszystkie serwisy platformy Node.js wykorzystują framework Express.js [12] w wersji 5.0. Rozwiązanie to charakteryzuje się minimalistyczną architekturą i niskim narzutem wydajnościowym. Framework nie narzuca sztywnej struktury katalogów (ang. *unopinionated framework*), co pozwala na elastyczne dostosowanie architektury każdego z mikroservisów do jego specyficznych wymagań. Express.js zapewnia również ustandaryzowane mechanizmy obsługi żądań pośredniczących (ang. *middleware*), routingu oraz zarządzania błędami.

3.3 Baza danych - Azure Cosmos DB

Azure Cosmos DB [8] to w pełni zarządzana baza NoSQL firmy Microsoft, oferująca globalną dystrybucję danych, elastyczne partycjonowanie oraz obsługę wielu modeli API

(SQL, MongoDB, Cassandra, Gremlin, Table). W projekcie AddiPi wykorzystano API SQL (Core), umożliwiające wykonywanie zapytań zbliżonych składnią do języka SQL na dokumentach zapisanych w formacie JSON.

Wybór Azure Cosmos DB został podyktowany następującymi czynnikami:

- brak konieczności zarządzania infrastrukturą serwera bazy danych dzięki wykorzystaniu modelu PaaS (Platform as a Service),
- elastyczny schemat dokumentów umożliwiający przechowywanie danych o różnej strukturze w ramach jednej bazy danych (addipi), z podziałem na odrębne kontenery (users, jobs, refresh-tokens),
- natywna integracja z ekosystemem Microsoft Azure,
- automatyczne indeksowanie wszystkich pól dokumentów.

3.4 Przechowywanie plików - Azure Blob Storage

Pliki G-code przesyłane przez użytkowników są przechowywane w usłudze Azure Blob Storage [7], w kontenerze gcode. Usługa ta zapewnia praktycznie nieograniczoną przestrzeń magazynową, relatywnie niskie koszty przechowywania danych oraz prosty interfejs umożliwiający przesyłanie i pobieranie obiektów binarnych.

3.5 Kolejowanie - Azure Service Bus

Azure Service Bus [10] to chmurowa usługa kolejowania komunikatów klasy enterprise firmy Microsoft. W systemie AddiPi pełni rolę brokera komunikatów pomiędzy usługami Files Service (producent komunikatów) oraz Queue Service (konsument).

Kolejka print-queue zapewnia semantykę dostarczania wiadomości typu at-least-once. Oznacza to, że w przypadku awarii usługi konsumenckiej komunikat pozostaje w kolejce i może zostać przetworzony powtórnie po ponownym uruchomieniu konsumenta.

3.6 Sterowanie urządzeniem - Azure IoT Hub

Azure IoT Hub [9] to usługa firmy Microsoft przeznaczona do zarządzania urządzeniami Internetu Rzeczy (IoT) w środowisku chmurowym. Umożliwia dwukierunkową komuni-

kację z urządzeniami (Device-to-Cloud oraz Cloud-to-Device), a także wywołanie metod bezpośrednich (Direct Methods), stanowiących synchroniczne wywołania RPC (zdalne wywołanie procedury) wykonane na urządzeniach komunikujących się za pośrednictwem protokołów MQTT lub AMQP.

W projekcie AddiPi usługa Printer Service wywołuje na agencie działającym na urządzeniu Raspberry Pi metody `startPrint`, `cancelPrint` oraz `getStatus`. Agent przesyła następnie telemetry w postaci zdarzeń, takich jak `print_started`, `print_progress` czy `print_completed`, które są odbierane przez usługę Printer Service za pośrednictwem endpointu kompatybilnego z Azure Event Hubs.

3.7 Frontend - React i Vite

Interfejs użytkownika został zbudowany z wykorzystaniem biblioteki React [6] 18 oraz języka TypeScript. Jako serwer deweloperski i narzędzie do budowania aplikacji wykorzystano Vite [17].

Do zarządzania stanem globalnym aplikacji zastosowano bibliotekę Zustand [14], umożliwiającą współdzielenie danych pomiędzy komponentami aplikacji w uproszczony sposób, bez konieczności implementowania rozbudowanej logiki zarządzania stanem.

Warstwa stylów została zrealizowana przy użyciu frameworka Tailwind CSS [16], który opiera się na wykorzystaniu niewielkich klas CSS reprezentujących pojedyncze właściwości stylów interfejsu użytkownika.

3.8 Kontenery - Docker i Docker Compose

Każdy serwis aplikacji posiada własny plik `Dockerfile` realizujący wieloetapowy proces budowania obrazu kontenera (*multi-stage build*). Rozwiązanie to pozwala na ograniczenie rozmiaru finalnych obrazów oraz oddzielenie środowiska kompilacji od uruchomieniowego.

Docker Compose [3] wykorzystano do zarządzania uruchamianiem wszystkich serwisów zarówno w środowisku lokalnym, jak i produkcyjnym. Narzędzie definiuje wspólną sieć `addipi-network` oraz konfigurację zmiennych środowiskowych odczytywanych z pliku `.env`.

3.9 CAD - SolidWorks

Projekt mechaniczny obudowy urządzenia wykonano w programie **SolidWorks** [2], stanowiącym środowisko do parametrycznego modelowania bryłowego oraz projektowania złożów mechanicznych. Oprogramowanie umożliwia tworzenie asocjatywnych modeli 3D, w których modyfikacja parametrów części powoduje automatyczną aktualizację złożów oraz dokumentacji technicznej.

Wybór programu SolidWorks był podyktowany dostępnością oprogramowania w ramach licencji studenckiej oraz szerokim wsparciem dla formatów eksportu modeli, takich jak .STEP (neutralny format wymiany modeli CAD), .STL (format wykorzystywany w procesie przygotowania modeli o druku 3D) oraz .3MF (format wymiany zawierający dodatkowe metadane procesu druku).

Projekt został podzielony na pliki .SLDPRT, reprezentujące pojedyncze części, oraz .SLDASM, definiujące kompletne złożenia mechaniczne.

3.10 Porównanie wybranych alternatyw

Tabela 1: Porównanie baz danych rozważanych w projekcie

Cecha	Cosmos DB	PostgreSQL	MongoDB Atlas
Model danych	Dokument (JSON)	Relacyjny	Dokument (BSON)
Zarządzanie	PaaS (bez serwera)	Samodzielny/PaaS	PaaS
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Elastyczny schemat danych	Tak	Nie	Tak
Dostępność bezpłatnego planu	Tak (400 RU/s)	Tak (self-hosted)	Tak (512 MB)

Tabela 2: Porównanie rozwiązań kolejkowania wiadomości

Cecha	Azure Service Bus	RabbitMQ	Apache Kafka
Model wdrożenia	PaaS	Self-hosted	Self-hosted / PaaS
Semantyka dostarczania	At-least-once	At-least-once	At-least-once
Złożoność konfiguracji	Niska	Średnia	Wysoka
Integracja z Azure	Natywna	Ograniczona	Pośrednia
Skalowalność	Automatyczna	Ręczna	Wysoka

4 Architektura systemu

4.1 Ogólna struktura

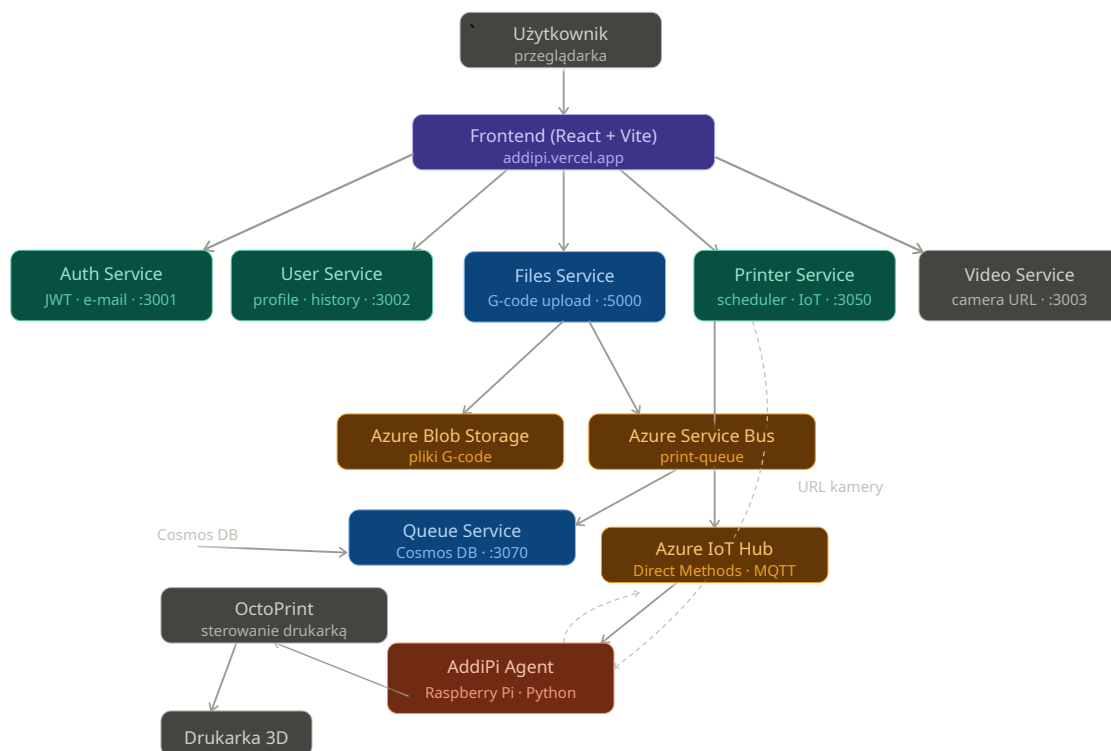
System AddiPi zbudowany jest w architekturze mikroserwisowej. Każdy serwis stanowi niezależną jednostkę wdrożeniową odpowiedzialną za określoną domenę biznesową oraz komunikuje się z pozostałymi komponentami za pośrednictwem zdefiniowanych interfejsów (HTTP REST lub kolejka komunikatów).

W ramach systemu wyodrębniono następujące komponenty:

1. **AddiPi Auth Service** (port 3001) - realizuje proces uwierzytelniania użytkowników oraz obsługę tokenów JWT.
2. **AddiPi User Service** (port 3002) - odpowiada za zarządzanie profilami użytkowników oraz historią zadań do wydruku.
3. **AddiPi Files Service** (port 5000) - umożliwia przesyłanie oraz przechowywanie plików G-code.
4. **AddiPi Queue Service** (porty 3070) - odbiera komunikaty z kolejki oraz zapisuje informacje o zadaniach w bazie Azure Cosmos DB.
5. **AddiPi Printer Service** (port 3050) - odpowiada za harmonogramowanie zadań do wydruku, komunikację z Azure IoT Hub oraz odbiór telemetrii urządzenia.
6. **AddiPi Video Service** (port 3003) - odpowiada za rejestrację i udostępnianie adresu URL kamery systemu OctoPrint.
7. **AddiPi Agent** (Raspberry Pi) - agent uruchamiany na urządzeniu Raspberry Pi, pośredniczący pomiędzy Azure IoT Hub a systemem OctoPrint.
8. **AddiPi Frontend** (Vercel / port 5173) - aplikacja webowa zbudowana z wykorzystaniem biblioteki React.

4.2 Diagram architektury

Na rysunku 3 przedstawiono sekwencyjny przepływ danych pomiędzy poszczególnymi komponentami zaprojektowanego systemu.



Rysunek 3: Diagram architektury systemu.

1. **Uwierzytelnianie użytkownika:** Użytkownik loguje się w systemie za pośrednictwem aplikacji klienckiej (frontendowej). Aplikacja nawiązuje komunikację z usługą *Auth Service*, która po pomyślnej weryfikacji tożsamości zwraca parę tokenów JWT (*JSON Web Token*).
2. **Przesyłanie pliku produkcyjnego:** Użytkownik przesyła plik z kodem sterującym (G-code). Aplikacja frontendowa wywołuje punkt końcowy (ang. *endpoint*) `POST /files/upload` w ramach usługi *Files Service*. Następnie komponent ten zapisuje plik w repozytorium *Azure Blob Storage* oraz publikuje komunikat `file_uploaded` w kolejce usługi *Azure Service Bus*.
3. **Inicjalizacja zadania w bazie danych:** Usługa *Queue Service* asynchronicznie odbiera komunikaty z kolejki i na ich podstawie tworzy dokument zadania w bazie danych *Azure Cosmos DB* (w ramach kontenera `jobs`), przypisując mu status początkowy `pending` lub `scheduled`.
4. **Harmonogramowanie i weryfikacja stanu urządzenia:** Usługa *Printer Service* w sposób cykliczny (z interwałem jednominutowym) weryfikuje dostępność zadań

oczekujących na realizację. Jeżeli urządzenie drukujące jest wolne, a zadanie spełnia kryteria rozpoczęcia procesu wytwórczego, status transakcji jest zmieniany na `waiting_for_printer_ready`. System przechodzi wówczas w stan oczekiwania na potwierdzenie gotowości operacyjnej drukarki.

5. **Inicjacja komunikacji IoT:** Po autoryzacji gotowości urządzenia przez użytkownika lub administratora, usługa *Printer Service* zdalnie wywołuje metodę bezpośrednią (ang. *Direct Method*) `startPrint` na dedykowanym agencie, wykorzystując do tego celu usługę *Azure IoT Hub*.
6. **Zarządzanie procesem drukowania przez agenta:** Komponent *AddiPi Agent* pobiera przypisany plik G-code z repozytorium *Azure Blob Storage*, przekazuje go bezpośrednio do interfejsu oprogramowania *OctoPrint* oraz inicjuje proces drukowania 3D. W trakcie pracy agent w sposób ciągły transmituje dane telemetryczne do usługi *Azure IoT Hub*, raportując bieżący postęp, status ukończenia lub ewentualne błędy krytyczne.
7. **Odbiór telemetry i aktualizacja stanu:** Usługa *Printer Service* konsumuje strumień danych telemetrycznych za pośrednictwem punktu końcowego zgodnego ze specyfikacją *Azure Event Hubs*, a następnie aktualizuje stan procesu w bazie *Azure Cosmos DB*.
8. **Odświeżanie danych w interfejsie:** Aplikacja kliencka (frontendowa) realizuje cykliczne zapytania sondujące (ang. *polling* z interwałem 5 sekund) do mikroserwisów, zapewniając bieżącą aktualizację stanu systemu prezentowanego użytkownikowi.

4.3 Model danych

W bazie danych *Azure Cosmos DB* dane są strukturyzowane i przechowywane w postaci dokumentów w formacie JSON. Zostały one pogrupowane w logicznych kontenerach, odpowiadających poszczególnym domenom biznesowym systemu.

4.3.1 Użytkownik (kontener users)

Dokument przechowuje informacje profilowe użytkowników, dane autoryzacyjne oraz status weryfikacji konta. Przykładową strukturę przedstawiono na listingu 1.

```
1  {
2    "id": "user_<timestamp>",
3    "email": "student@uwr.edu.pl",
4    "firstName": "Jan",
5    "lastName": "Kowalski",
6    "password": "<brypt hash>",
7    "role": "user",
8    "isVerified": true,
9    "verificationToken": "<hex>",
10   "verificationTokenExpiry": "2025-01-01T00:00:00Z",
11   "createdAt": "2025-01-01T00:00:00Z",
12   "updatedAt": "2025-01-01T00:00:00Z"
13 }
```

Listing 1: Przykładowy dokument użytkownika w Azure Cosmos DB

Pole role definiuje uprawnienia w systemie i może przyjmować wartość user (użytkownik standardowy) lub admin (administrator systemu).

4.3.2 Zadanie do wydruku (kontener jobs)

Dokument zadania (listing 2) agreguje informacje o pliku sterującym, powiązonym użytkowniku, metrykach czasu oraz bieżącym stanie realizacji procesu produkcyjnego.

```
1  {
2    "id": "<timestamp string>",
3    "fileId": "20251203_014648_model.gcode",
4    "originalFileName": "model.gcode",
5    "userId": "user_<timestamp>",
6    "userEmail": "student@uwr.edu.pl",
7    "status": "printing",
8    "scheduledAt": "2025-12-01T15:00:00Z",
9    "createdAt": "2025-11-30T10:00:00Z",
10   "startedAt": "2025-12-01T15:01:00Z",
11   "completedAt": "2025-12-01T17:30:00Z",
12   "progress": 87.5,
```

```

13   "printTime": 4500,
14   "printTimeLeft": 600,
15   "deviceId": "raspberrypi-mkt-01",
16   "failureReason": null
17 }

```

Listing 2: Przykładowy dokument zadania do wydruku w Cosmos DB

Pole `status` może przyjmować wartości: `pending`, `scheduled`, `waiting_for_printer_ready`, `delayed`, `printing`, `completed`, `failed`, `cancelled`.

4.3.3 Token odświeżania (kontener `refresh-tokens`)

W celu zapewnienia bezpiecznej, bezstanowej autoryzacji sesji, tokeny odświeżania (ang. *refresh tokens*) są utrwalane w dedykowanym kontenerze (listing 3).

```

1  {
2    "id": "<userId>_<timestamp>",
3    "userId": "user_<timestamp>",
4    "token": "<JWT string>",
5    "expiresAt": "2025-12-08T00:00:00Z",
6    "createdAt": "2025-12-01T00:00:00Z"
7  }

```

Listing 3: Przykładowy dokument refresh tokena

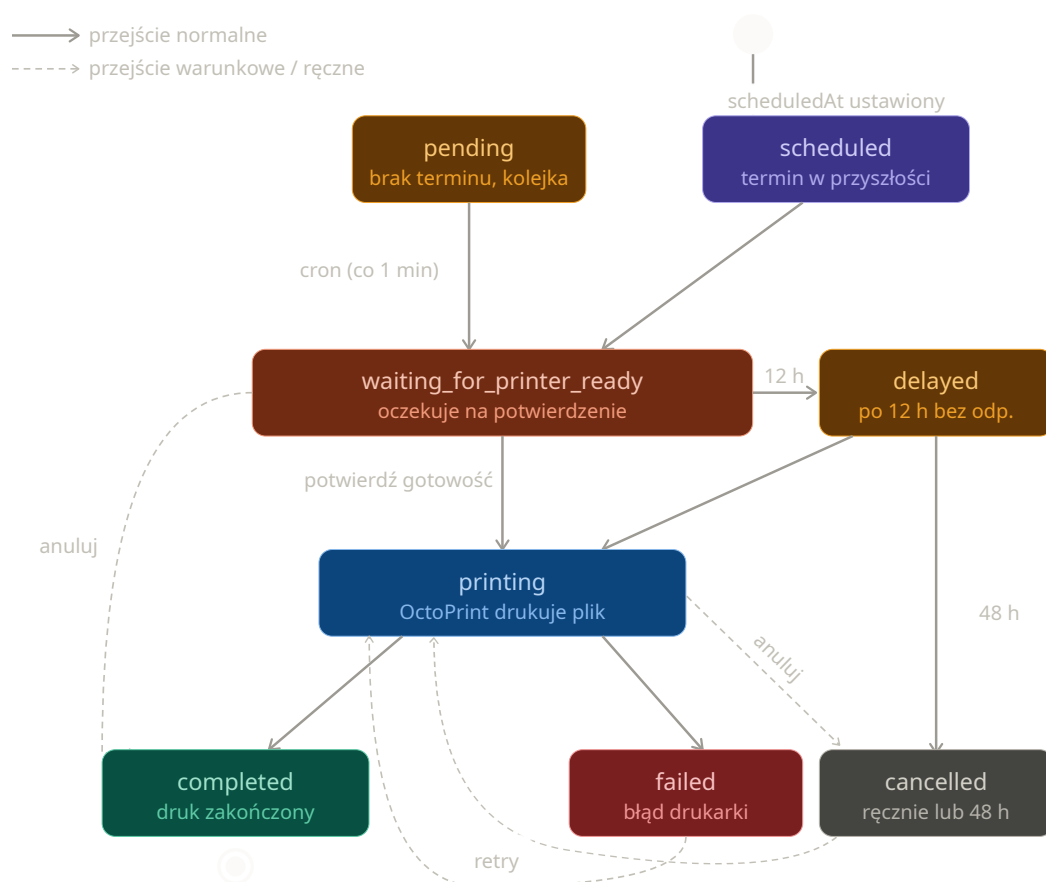
4.4 Cykl życia zadania do wydruku

Zadanie w procesie przetwarzania i realizacji może znajdować się w jednym z następujących stanów:

- **pending** — zadanie bez określonego terminu realizacji, oczekujące w kolejce ogólnej systemu.
- **scheduled** — zadanie posiadające ściśle zaplanowany termin realizacji, oczekujące na osiągnięcie wskazanego czasu rozpoczęcia. Po nadejściu zaplanowanego czasu zadanie to otrzymuje najwyższy priorytet wykonania.
- **waiting_for_printer_ready** — usługa harmonogramująca (ang. *scheduler*) wybrała zadanie do realizacji; system oczekuje na manualne potwierdzenie gotowości urządzenia przez użytkownika.

- **delayed** — stan opóźnienia, w którym system oczekuje na potwierdzenie gotowości drukarki dłużej niż 3 h; do użytkownika wysyłane jest automatyczne powiadomienie przypominające.
- **printing** — proces drukowania modelu sterującego jest aktywnie realizowany przez oprogramowanie *OctoPrint*.
- **completed** — proces drukowania 3D został pomyślnie sfinalizowany.
- **failed** — w trakcie realizacji procesu produkcyjnego wystąpił błąd krytyczny uniemożliwiający dalsze drukowanie.
- **cancelled** — zadanie zostało anulowane ręcznie przez użytkownika bądź administratora lub automatycznie przez system po upływie 48 h oczekiwania na potwierdzenie gotowości urządzenia.

Na rysunku 4 przedstawiono formalny model maszyny stanów, obrazujący dopuszczalne przejścia fazowe w cyklu życia zadania produkcyjnego.



Rysunek 4: Diagram maszyny stanów cyklu życia zadania do wydruku.

4.5 Bezpieczeństwo i uwierzytelnianie

W celu zapewnienia poufności danych oraz kontroli dostępu, w systemie zaimplementowano dwutokenowy model uwierzytelniania oparty na standardzie JWT (*JSON Web Token*). Mechanizm ten wykorzystuje dwa rodzaje obiektów:

- **Access token** — token dostępu o krótkim czasie autoryzacji (ważny przez 15 minut), przekazywany w nagłówku protokołu HTTP (`Authorization: Bearer <token>`). Integralność i autentyczność tokenu są weryfikowane kryptograficznie przy użyciu klucza symetrycznego `JWT_SECRET`.
- **Refresh token** — token odświeżający o wydłużonym czasie ważności (wynoszącym 7 dni), utrwalany w bazie danych *Azure Cosmos DB* oraz przechowywany po stronie aplikacji klienckiej. Podpisywany jest przy użyciu dedykowanego klucza `JWT_REFRESH_SECRET`. Służy do generowania nowej pary tokenów bez konieczności ponownego wprowadzania danych uwierzytelniających przez użytkownika.

Hasła użytkowników nie są przechowywane w bazie danych w postaci jawnej. Do ich zabezpieczenia wykorzystano algorytm funkcjonału skrótu *bcrypt* z parametrem kosztu (ang. *salt rounds*) ustawionym na wartość 10.

Proces rejestracji nowych kont został ze względów bezpieczeństwa ograniczony wyłącznie do adresów poczty elektronicznej w domenie akademickiej `@uwr.edu.pl`. Pełna aktywacja profilu wymaga weryfikacji dwuetapowej poprzez kliknięcie w unikalny link aktywacyjny przesłany na adres e-mail użytkownika.

Kontrola dostępu na poziomie kodu realizowana jest za pomocą oprogramowania pośredniczącego (ang. *middleware*):

- `requireAuth` — odpowiada za ekstrakcję i weryfikację tokenu dostępu poprzez wysłanie wewnętrznego żądania weryfikacyjnego do punktu końcowego `POST /auth/verify` usługi *Auth Service*.
- `requireAdmin` — realizuje mechanizm kontroli dostępu opartej na rolach (ang. *Role-Based Access Control*, RBAC), sprawdzając, czy zdekodowany profil użytkownika posiada przypisane uprawnienia klasy `admin`.

5 Implementacja serwisów backendowych

5.1 Auth Service

5.1.1 Opis ogólny

5.1.2 Endpointy API

5.1.3 Kluczowe fragmenty implementacji

5.1.4 Weryfikacja e-mail

5.2 User Service

5.2.1 Opis ogólny

5.2.2 Endpointy API

5.2.3 Mechanizm autoryzacji

5.3 Files Service

5.3.1 Opis ogólny

5.3.2 Przepływ przesyłania pliku

5.4 Queue Service

5.4.1 Opis ogólny

5.4.2 Listener kolejki

5.4.3 Endpointy HTTP API

5.5 Printer Service

5.5.1 Opis ogólny

5.5.2 Harmonogram

5.5.3 Monitor gotowości

5.5.4 Listener telemetrii

5.5.5 Enpointy HTTP API

5.6 Video Service

6 Implementacja frontendu

6.1 Struktura aplikacji

6.2 Klient API

6.3 Routing i ochrona tras

6.4 Ekran aplikacji

6.4.1 Strona główna

6.4.2 Dashboard użytkownika

6.4.3 Upload i planowanie

6.4.4 Kontrola druku

6.4.5 Panel administracyjny

6.5 Internacjonalizacja

6.6 Obsługa motywu

7 Agent Raspberry Pi - AddiPi Agent

7.1 Opis ogólny

7.2 Architektura agenta

7.3 Połączenie z IoT Hub

7.4 Obsługiwane metody bezpośrednie

7.5 Przebieg obsługi metody startPrint

7.6 Telemetria

7.7 Komunikacja z OctoPrint

8 Obudowa urządzenia - projekt CAD

8.1 Cel i zakres projektu mechanicznego

8.2 Moduł CASE - obudowa Raspberry Pi z kamerą

8.2.1 Podstawa projektowa

8.2.2 Zakres modyfikacji

8.2.3 Mocowanie kamery

8.2.4 Struktura plików CASE

8.3 Moduł STAND - podstawka regulowana

8.4 Złożenie całości

8.5 Parametry wydruku

8.6 Instrukcja montażu

9 Konfiguracja Raspberry Pi

9.1 Opis środowiska

9.2 Instalacja agenta

9.3 Automatyczne uruchamianie agenta przez systemd

9.4 Tunel Cloudflare - ekspozycja kamery

9.4.1 Instalacja cloudflared

9.4.2 Skrypt publikowania URL kamery

9.4.3 Cykliczne odnawianie URL

9.5 Konfiguracja crontab

9.5.1 Uzasadnienie parametrów

9.6 Problem z dostępnością sieci przy starcie

9.7 Podsumowanie konfiguracji Raspberry Pi

10 Wdrożenie i infrastruktura

10.1 Infrastruktura chmurowa

10.2 Inicjalizacja infrastruktury

10.3 Konteneryzacja - Dockerfiles

10.4 Docker Compose

10.5 Zmienne środowiskowe

10.6 Wdrożenie frontendu

10.7 Wdrożenie agenta

11 Testowanie systemu

11.1 Metodologia testowania

11.2 Testy jednostkowe - Auth Service

11.3 Testy integracyjne przez Postman

11.4 Test end-to-end - pełny przepływ druku

11.5 Testy wydajnościowe i obserwacje

12 Podsumowanie

12.1 Osiągnięcia

12.2 Napotkane trudności

12.3 Możliwości rozwoju

12.4 Wnioski

Literatura

- [1] Alexandre Joel Chorin. Numerical solution of the navier-stokes equations. *Mathematics of computation*, 22(104):745–762, 1968.
- [2] Dassault Systèmes. SOLIDWORKS – 3D CAD Design Software. <https://www.solidworks.com>. Dostęp: 2026.
- [3] Docker Inc. Docker Documentation. <https://docs.docker.com>. Dostęp: 2026.
- [4] Gina Häußge. Octoprint – The snappy web interface for your 3D printer. <https://octoprint.org>. Dostęp: 2026.
- [5] Maciej Matyka, Arzhang Khalili, and Zbigniew Koza. Tortuosity-porosity relation in porous media flow. *Physical Review E*, 78(2):026306, 2008.
- [6] Meta Platforms, Inc. React – The library for web and native user interfaces. <https://react.dev>. Dostęp: 2026.
- [7] Microsoft Corporation. Azure Blob Storage documentation. <https://learn.microsoft.com/en-us/azure/storage/blobs/>. Dostęp: 2026.
- [8] Microsoft Corporation. Azure Cosmos DB documentation. <https://learn.microsoft.com/en-us/azure/cosmos-db/>. Dostęp: 2026.
- [9] Microsoft Corporation. Azure IoT Hub documentation. <https://learn.microsoft.com/en-us/azure/iot-hub/>. Dostęp: 2026.
- [10] Microsoft Corporation. Azure Service Bus documentation. <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>. Dostęp: 2026.
- [11] Microsoft Corporation. Typescript – JavaScript With Syntax For Types. <https://www.typescriptlang.org>. Dostęp: 2026.
- [12] OpenJS Foundation. Express – Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>. Dostęp: 2026.
- [13] OpenJS Foundation. Node.js – JavaScript runtime built on Chrome’s V8 engine. <https://nodejs.org>. Dostęp: 2026.

- [14] Poimandres. Zustand – Bear necessities for state management in React. <https://github.com/pmndrs/zustand>. Dostęp: 2026.
- [15] Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>. Dostęp: 2026.
- [16] Tailwind Labs Inc. Tailwind CSS – Rapidly build modern websites without ever leaving your HTML. <https://tailwindcss.com>. Dostęp: 2026.
- [17] Evan You et al. Vite – Next Generation Frontend Tooling. <https://vitejs.dev>. Dostęp: 2026.

Załączniki

A Konfiguracja środowiska deweloperskiego

A.1 Wymagania

A.2 Pierwsze uruchomienie

B Struktura repozytoriów

C Endpointy API - podsumowanie